10-bit SAR ADC Design in Sky130 – Project Report

Contents

1 Introduction and Specifications

The successive-approximation (SAR) ADC is the most commonly deployed topology of analog-to-digital converter. Owing to its low power consumption, low latency and compact form factor, it has proven supremely versatile, with notable usage in IoT devices, medical equipment and microcontrollers. This document describes an original 10-bit SAR ADC designed and verified in the SkyWater Sky130 open-source 130nm process — a CMOS PDK developed by SkyWater Technology and Google, providing full device model access without NDA — targeting the specifications in Table 1. The design covers the complete front-end flow: behavioral modeling from first principles, transistor-level schematic capture in xschem, ngspice simulation against Sky130 device models, and Verilog RTL synthesized to Sky130 standard cells via Yosys. Physical layout is out of scope; the project targets schematic-level verification.

Table 1: Design Specifications

Parameter	Value
Resolution	10 bits
Sample Rate	1 $\frac{\text{MS}}{\text{s}}$
ENOB	$\geq$ 9 bits
Supply Voltage, V_{DD}	1.8 V
Reference Voltage, V_{ref}	1.8 V
Input Range	$0 - 1.8\,\text{V}$
Process	SkyWater Sky130, 130nm

The design followed a bottom-up flow, progressing sequentially through each of the following stages:

1. First-principles sizing and yield analysis via Python behavioral model
2. Block-level schematic capture in xschem and isolation verification in ngspice
3. Verilog RTL synthesis to Sky130 standard cells via Yosys
4. Full-ADC SPICE integration and static performance characterisation

This document reports the design decisions, block-level verification, integration debugging and static characterisation of the 10-bit SAR ADC. Ultimately, the design achieves a static ENOB of 7.27 bits, falling short of its 9-bit target, attributable to a characterised top-plate feedthrough mechanism, detailed in Section 4.

2 Architecture Overview

2.1 Algorithm Description

The binary code corresponding to the input voltage V_{in} is determined through a recursive algorithm with the high or low character of each bit being resolved individually. The algorithm works by treating each code as a sum of weighted contributions of V_{ref}, where bit N contributes a weight of $\frac{1}{2^{10-N}} \cdot V_{ref}$ to the overall trial voltage V_{analog}. A bit is kept if its voltage contribution keeps V_{analog} below V_{in}, otherwise it is rejected. This process repeats until all 10 bits are resolved.

Using an example voltage of 1.30 V, the behavior of the algorithm can be traced through as follows:

1. Assert bit 9 of the trial code, corresponding to an analog voltage of $V_{\text{analog}} = \frac{1}{2^{10-9}} \cdot V_{ref} = 0.9\,\text{V}$

2. Compare $V_{in} = 1.3\,\text{V}$ to V_{analog}; $V_{in} > V_{\text{analog}}$, therefore retain bit 9, assert bit 8. New trial code: 1100000000, corresponding voltage $V_{\text{analog}} = \frac{1}{2^{10-9}} \cdot V_{ref} + \frac{1}{2^{10-8}} \cdot V_{ref} = 1.35\,\text{V}$

3. Compare V_{in} to V_{analog}; $V_{in} < V_{\text{analog}}$ therefore reject bit 8, assert bit 7. New code = 1010000000, corresponding voltage $V_{\text{analog}} = \frac{1}{2^{10-9}} \cdot V_{ref} + \frac{1}{2^{10-7}} \cdot V_{ref} = 1.125\,\text{V}$

This process repeats until all 10 bits are individually checked, in this case resulting in the code $1011100010 = 1.297\,\text{V}$.

2.2 Operation

The design is comprised of four major blocks: the bootstrapped switch, CDAC binary-weighted array, StrongARM comparator and SAR logic block, and works in two phases: sampling and conversion. During the sampling phase, the bootstrapped switch connects V_{in} to all bottom plates of the CDAC array simultaneously, while the top plate is reset to $V_{cm} = V_{ref}\,/\,2$. This establishes a charge on the top plate node of $Q = (V_{ref}/2 - V_{in}) \cdot C_{total}$. At the start of conversion, the bootstrapped switch is disconnected, leaving the top plate floating with its charge preserved while the bottom plate of each node is shorted to ground. The SAR logic then performs a binary search from the MSB downward: each trial bit steers its corresponding bottom plate to V_{ref}, shifting V_{DAC} upward by an amount proportional to that capacitor's weight. At the conclusion of this search, the StrongARM comparator evaluates whether $V_{ref}/2$ is greater than V_{DAC}, equivalent to testing whether V_{in} exceeds the analog value of the current trial code. The SAR logic retains or clears each bit accordingly:

1. Assert trial code 1000000000; bottom plate of $512C$ switches to V_{ref}, shifting V_{DAC} upwards to $\frac{512}{1024}V_{ref}$.
2. If $V_{DAC} < V_{ref}/2$ (equivalently, $V_{in} > \frac{512}{1024}V_{ref}$), retain bit 9; otherwise clear it and return that bottom plate to ground
3. Assert the next trial bit (bit 8; adding an upwards shift of $\frac{256}{1024}V_{ref}$ to V_{DAC}) and repeat in descending order until all 10 bits are resolved

This process is summarized by the block diagram in figure 1.

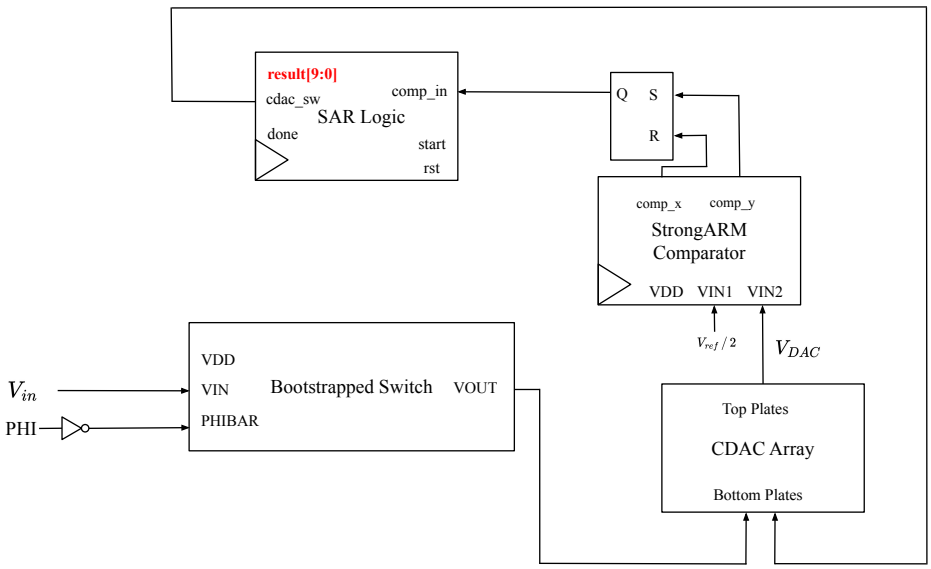

Figure 1: Block diagram of SAR ADC

3 Block Design and Sizing

3.1 Python Behavioral Model

The minimum valid unit capacitance, C_{unit}, that can be used for any capacitor in the CDAC array is governed by the thermal noise floor in equation 1.

$$C_{min} = \frac{kT}{V_{LSB}^2} \tag{1}$$

with $V_{LSB} = \frac{V_{ref}}{2^N}$, where N is the resolution of the instrument. Substituting $k = 1.38 \times 10^{-23}\,\mathrm{J \cdot K}$, $T = 300\,\mathrm{K}$, $V_{ref} = 1.8\,\mathrm{V}$ and $N = 10$, C_{min} is found as $5.36\,\mathrm{fF}$.

In order to determine a suitable unit capacitance, a Monte Carlo mismatch analysis was conducted. In each trial, C_{unit} was swept from $5\,\mathrm{fF}$ to $54\,\mathrm{fF}$ and a binary-weighted

capacitor array, $[512, 256, 128, ..., 2, 1, 1] \times C_{unit}$ was generated, representing the CDAC array. Each capacitor in this array was perturbed according to $\sigma_{C/C} \,/\, \sqrt{n_{cells}}$, where the mismatch $\sigma_{C/C}$ is 2%, and $n_{cells} = C_{unit} \,/\, C_{min}$. Using this representation of the CDAC array, the SAR conversion algorithm was executed on $2^{16} = 65\,536$ input voltages, 64 per binary code. For each trial, the DNL and INL were computed and the maximum INL was recorded. This process was repeated for 10 000 trials.

The yield is computed as the fraction of trials where the maximum INL is less than 0.5 LSB. As can be seen in figure 2, the yield surpasses its target of 99% at approximately 24 fF. Therefore, to allow for some margin, a target value of $C_{unit} = 25\,\text{fF}$ was selected, giving a total CDAC capacitance of $1024 \times 25\,\text{fF} = 25.6\,\text{pF}$.

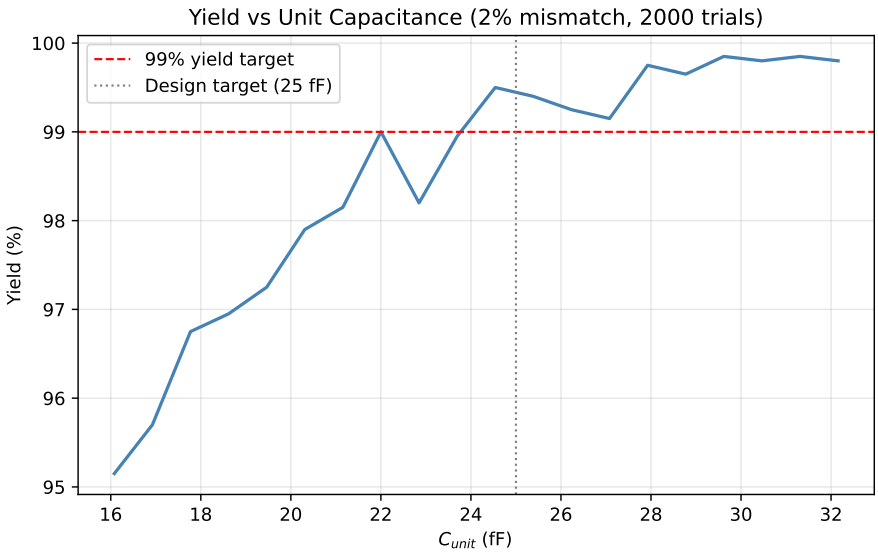

Figure 2: Yield vs. C_{unit} sweep across 10 000 Monte Carlo trials at $\sigma_{C/C} = 2\%$.

To validate the selected $C_{unit} = 25\,\text{fF}$, a further 10 000-trial Monte Carlo simulation was run at this fixed value. Figure 3 shows the resulting distribution of maximum INL across all trials. The distribution is centred near 0.2 LSB, with a tail that barely reaches the 0.5 LSB threshold, confirming a yield of 99.2%.

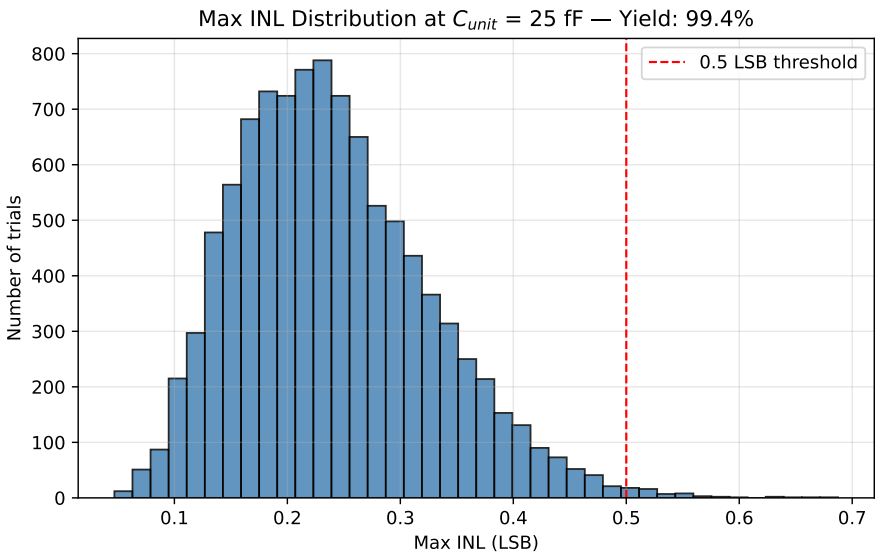

Figure 3: Distribution of maximum INL at $C_{unit} = 25\,\text{fF}$, 10 000 Monte Carlo trials, $\sigma_{C/C} = 2\%$. Yield: 99.2%.

Figure 4 shows the DNL and INL of a representative single trial at $C_{unit} = 25\,\text{fF}$. The DNL remains well within $\pm 0.5\,\text{LSB}$ across all 1024 output codes, with no missing codes. The INL exhibits a random-walk character, as expected from independent capacitor mismatch errors accumulating across the code range, and stays within $\pm 0.5\,\text{LSB}$ throughout.

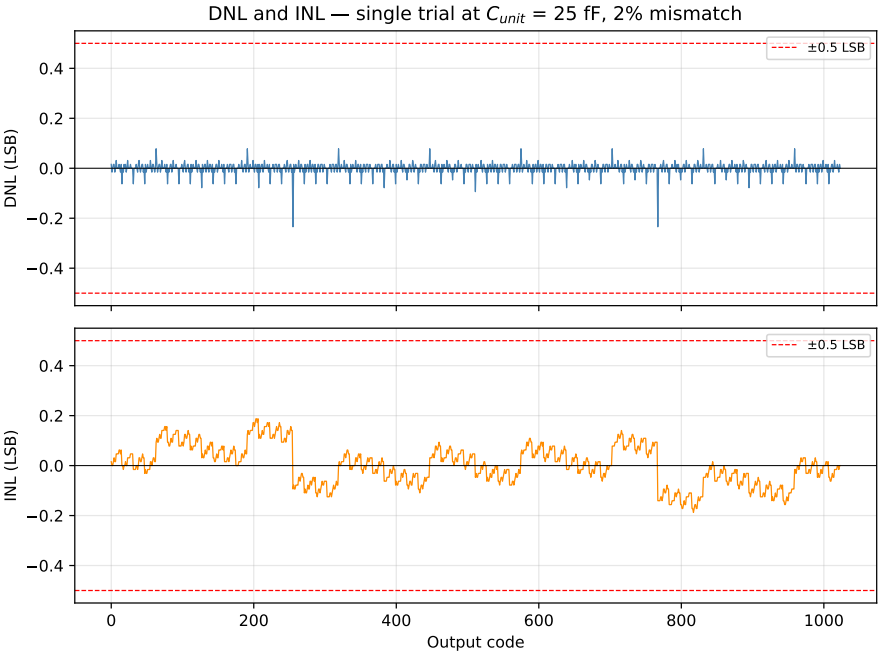

Figure 4: DNL and INL for a single representative trial at $C_{unit} = 25\,\text{fF}$, $\sigma_{C/C} = 2\%$.

3.2 CDAC

The CDAC is a binary-weighted array of capacitors, $[512, 256, 128, ..., 2, 1, 1] \times C_{unit}$. Including the termination capacitor of capacitance C_{unit}, it spans 11 capacitors with a total capacitance of $1024 \cdot C_{unit} = 25.6\,\text{pF}$. The top plate of each capacitor is tied to the common node V_{DAC}, while each capacitor has a unique bottom plate node. During operation, the CDAC undergoes two stages:

1. Sampling: the bottom plate of each capacitor is driven to V_{in}, while V_{DAC} is driven to $V_{cm} = V_{ref}\,/\,2$. This locks in a charge of $Q = 1024 \cdot C_{unit} \cdot (V_{ref}\,/\,2 - V_{in})$ onto the CDAC.

2. Conversion: V_{DAC} is disconnected from its source and becomes floating. The SAR logic block drives the bottom plate of each capacitor to either V_{ref} or ground, depending on whether the corresponding bit in the code under trial is high or low. For example, if the code under trial is 1010000000, then the bottom plates of $C_9 = 512C_{unit}$ and $C_7 = 128C_{unit}$ are sent to V_{ref}, with the remaining bottom plates being shorted to ground. At the end of conversion, $V_{DAC} = \frac{W}{1024}V_{ref} + V_{ref}\,/\,2 - V_{in}$, where W is the sum of the weights of each capacitor whose bottom plate is at $V = V_{ref}$. Note that during conversion, the bottom plate of the termination capacitor is always shorted to ground.

For each analog voltage under test, sampling occurs once at the start of the procedure, while conversion happens once per bit, progressing from the MSB to the LSB. The transistor-level logic that controls the CDAC's behavior is shown in figure 5.

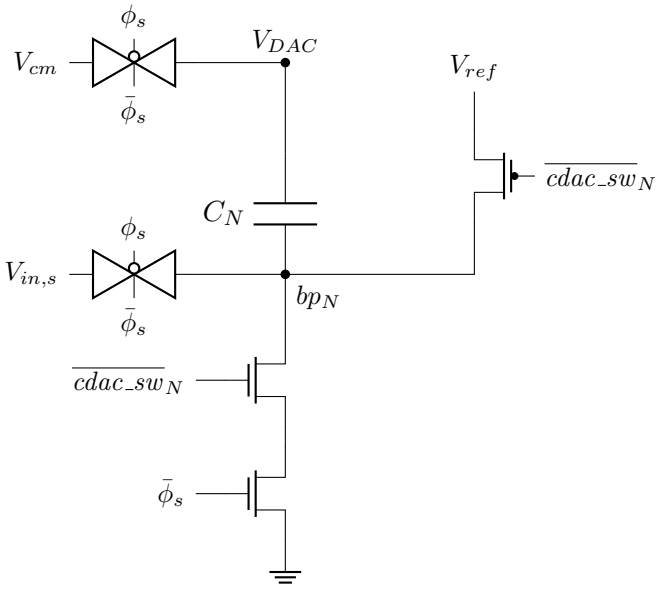

Figure 5: Transistor-level design of one CDAC cell

The diagram in figure 5 applies to all capacitors in the CDAC except for the termination capacitor. Note that for compactness, the transmission gate responsible for charging

V_{DAC} to V_{cm} during sampling is included in figure 5; however, because V_{DAC} is shared by all capacitors, the entire CDAC only uses one such transmission gate rather than one per cell. The signal $cdac\_sw_N$ represents the high/low value of N^{th} trial bit in the code under test. The capacitor C_N is responsible for the voltage contribution of this bit to V_{DAC}, and has a capacitance value of $C_N = 2^N \cdot C_{unit}$.

During sampling, ϕ_s is high, meaning that both transmission gates are active and V_{DAC} gets sent to V_{cm} while each V_{bp_N} is sent to V_{in}. Thereafter, ϕ_s turns off and both transmission gates are switched off, leaving V_{DAC} floating and initiating the conversion stage. If $cdac\_sw_N$ is high, then its complement $\overline{cdac\_sw_N}$ is low and the PMOS of the pull-up network turns on, shorting bp_N to V_{ref}. Otherwise, the top NMOS in the pull-down network is activated and bp_N is routed to ground. The additional NMOS gated by $\bar{\phi}_s$ in the pull-down network is introduced due to contention; if it were not there, then the pull-down network would provide a path to ground during sampling while $cdac\_sw_N$ is low. Since ϕ_s is guaranteed to be high during sampling and low during conversion, this NMOS ensures that the pull-down network is never active during sampling but can still be active during conversion.

Note that due to its repetitive nature, no schematic for the CDAC array was designed by hand, with `cdac.spice` instead being written directly.

3.3 Bootstrapped Switch

3.3.1 Motivation

Bootstrapping is necessary in order to avoid bandwidth modulation and harmonic distortion when charging the bottom plates of the CDAC up to V_{in}. When an NMOS is used to charge up the CDAC, the channel resistance R_{on} of the FET and the capacitance of the CDAC create a low-pass filter with cutoff frequency given by equation 2.

$$f_n = \frac{1}{2\pi R_{on} C_{DAC}} \tag{2}$$

The channel resistance R_{on} of an NMOS is given by equation 3.

$$R_{on} = \frac{1}{\mu_n C_{ox} \frac{W}{L} (V_{GS} - V_{th,n})} \tag{3}$$

If the FET is gated by V_{DD}, then $V_{GS} = V_{DD} - V_{in}$, meaning that if V_{in} changes, the bandwidth of the low-pass filter varies, attenuating signals inconsistently. This unevenness results in harmonic distortion, lowering the effective resolution of the instrument. The bootstrapped switch combats this by lifting the gate voltage of the FET to $V_{DD} + V_{in}$, therefore fixing V_{GS} at V_{DD}, thus fixing R_{on} and the corresponding cutoff frequency of the transfer function.

3.3.2 Topology and Operation

The topology of the bootstrapped switch is shown in figure 6. It operates in two stages.

1. When ϕ_s is low, the PMOS Q_2 is off while the NMOS Q_5 is on, therefore wiring the node X to ground. As such, the NMOS's Q_4 and Q_0 are off, while the PMOS Q_1 and the NMOS Q_3 are on. Therefore, there is a direct pathway from V_{DD} to ground through C_{boot}, which charges up to V_{DD}.

2. When ϕ_s turns on, transistors Q_3 and Q_5 turn off while transistor Q_2 turns on, allowing $V_X = V_{DD} + V_{in}$. This causes transistor Q_1 to turn off and transistors Q_4 and Q_0 to turn on, thus providing a pathway for V_{in} to flow into $V_{in,s}$ with the gate of Q_0 fixed at V_X.

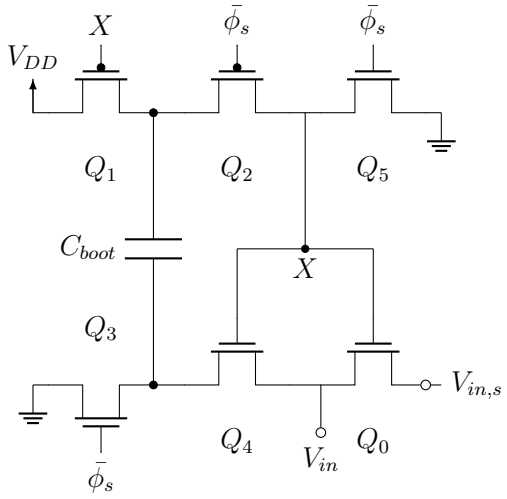

Figure 6: Bootstrapped sampling switch. Q_0 is the main sampling device; C_{boot} is precharged to V_{DD} during $\bar{\phi}_s$ and bootstraps the gate of Q_0 to $V_{in} + V_{DD}$ during ϕ_s.

3.3.3 Parameter Derivations

Together with the CDAC, the bootstrapped switch enforces the three-phase behavior of the signal ϕ_s. During phase 1, ϕ_s must be low so as to allow C_{boot} to charge. Then, ϕ_s must become high to turn the switch on and charge up the CDAC. Finally, ϕ_s permanently turns off, disconnecting the CDAC from the bootstrapped switch which has accomplished its purpose by fixing charge onto the CDAC.

In totality, this means that ϕ_s must be a pulse which must start at a delay. The length of this pulse determines the sizing of the transistors and the value of C_{boot}. This length, in turn, is determined by the sample rate design target, 1 megasample per second.

A rate of 1 megasample per second implies a period of 1 µs. Using a typical 70-30 split, with 30% of the period allotted to sampling, the width of the pulse is 300 ns. During the

width of this pulse, the CDAC must be charged up enough to render an error due to rounding impossible. Practically, this means that the error of the CDAC's voltage must be less than half of an LSB. The error of the CDAC's voltage, defined as the difference between V_{in} and V_{DAC}, is given by equation 4.

$$\text{Error} = V_{in} \cdot e^{-t\,/\,R_{on}C_{DAC}} \tag{4}$$

Requiring this error to be less than the value of half of one LSB, Error $< \frac{V_{ref}}{2^{N+1}}$, and assuming a worst-case scenario of $V_{in} = V_{ref}$, an expression for R_{on} can be derived.

$$R_{on} \leq \frac{t}{C_{DAC} \cdot 2^{N+1} \cdot \ln 2} \tag{5}$$

Substituting $t = 300\,\text{ns}$, $C_{DAC} = 25.6\,\text{pF}$ and $N = 10$ into equation 5, R_{on} is found as $1.54\,\text{k}\Omega$. Applying the channel resistance equation of equation 3, with $V_{GS} = V_{DD} = 1.8\,\text{V}$ due to bootstrapping, $\mu_n C_{ox} = 270\,\frac{\mu\text{A}}{\text{V}^2}$ as set by Sky130, and $l = 150\,\text{nm}$ (the minimum allowable channel length in Sky130), the minimum value for w is found as $273.8\,\text{nm}$. Allowing for an ample margin, the transistor Q_0 was chosen to have a width of $1\,\mu\text{m}$. Auxiliary transistors Q_1 and Q_{3-5} were sized to match Q_0's width layout for convenience, while Q_2 was sized conservatively with a width double that of Q_0. All transistors have an identical length of $150\,\text{nm}$.

Using these design parameters, a value of C_{boot} was selected that sufficiently minimizes droop. The voltage droop on C_{boot} due to charge sharing with the parasitic gate capacitance of Q_0, C_{gate}, is given by equation 6.

$$\frac{\Delta V}{V_{DD}} = \frac{C_{gate}}{C_{gate} + C_{boot}} \tag{6}$$

Setting this quantity to be strictly less than 0.01,

$$\begin{aligned} \frac{C_{gate}}{C_{gate} + C_{boot}} &< 0.01 \\ 100 \cdot C_{gate} &< C_{boot} + C_{gate} \\ \boxed{C_{boot} > 99 \cdot C_{gate}} & \end{aligned} \tag{7}$$

The gate capacitance, C_{gate}, is given by $C_{gate} = wl \cdot C_{ox}$. Substituting $w = 1\,\mu\text{m}$, $l = 150\,\text{nm}$ and $C_{ox} = 17\,\frac{\text{fF}}{\mu\text{m}^2}$ as set by Sky130, $C_{boot} > 252.45\,\text{fF}$. To significantly exceed this mark, C_{boot} was chosen as $500\,\text{fF}$. The final design choices are summarized in table 2.

Table 2: Sizing Choices

Parameter	Value
Width of Q_0	1 µm
Channel length of Q_0	150 nm
C_{boot}	500 fF

3.4 StrongARM Comparator

The StrongARM comparator is the block which performs the bitwise retain or reject decision making for each trial code. Its topology is shown in figure 7.

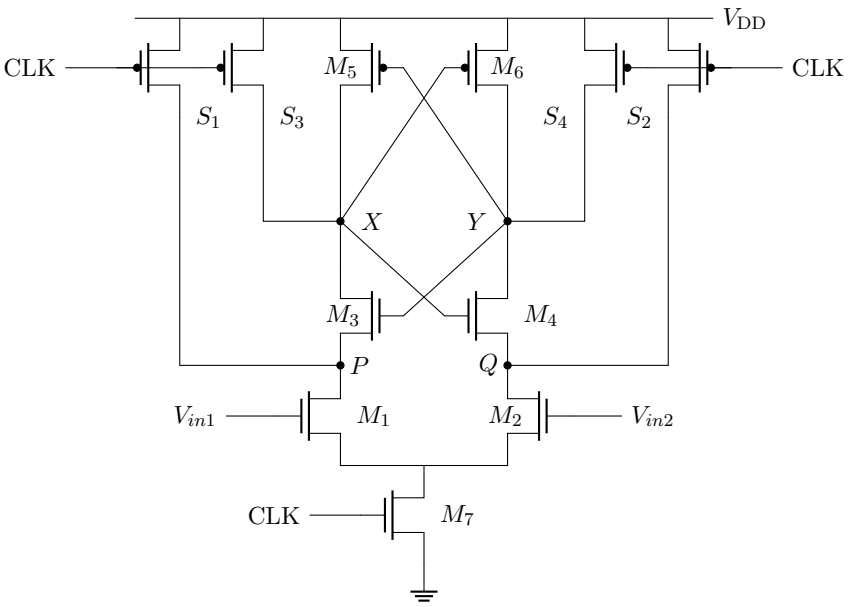

Figure 7: StrongARM latch comparator (Figure 1(b) from Razavi [**?**]). M1–M2 form the input differential pair, M3–M4 the cross-coupled NMOS latch, M5–M6 the cross-coupled PMOS latch, M7 the tail switch, and S1–S4 the precharge switches. Nodes P and Q are the intermediate amplification nodes; X and Y are the differential outputs.

When the clock is low, the pull-up network is active and the nodes P, Q, X and Y are pulled up to V_{DD}. When the clock turns on, the pull-up network is disabled while the NMOS pull-down network comprised of M1, M2 and M7 is enabled, offering a pathway for the voltages at P and Q to discharge to ground. If V_{in1} exceeds V_{in2}, the gate-source voltage and hence the rate of discharge at P is greater than that of Q, and if the reverse is true ($V_{in2} > V_{in1}$), then Q discharges faster than P. Suppose that $V_{in1} > V_{in2}$. Then, P discharges faster than Q, meaning that the NMOS M3 turns on before M4, offering a discharge pathway to ground for the voltage at X. Due to the cross-coupling between M3, M4, M5 and M6, X decreasing means that the gate voltage at M4 decreases, slowing the rate of discharge at Y. Eventually, V_X is low enough to enable M6 while disabling M4, hence pulling Y up to V_{DD}. Therefore, if $V_{in1} > V_{in2}$, V_Y settles at V_{DD} at steady state,

while V_X settles at 0. Following identical logic, in the reverse case where $V_{in2} > V_{in1}$, V_Y settles at 0 while V_X settles at V_{DD}.

During operation, $V_{in1} = V_{ref} / 2$ while $V_{in2} = V_{DAC}$. As noted in subsection 3.2, $V_{DAC} = \frac{W}{1024} V_{ref} + V_{ref} / 2 - V_{in}$ at the end of conversion. The condition for Y to be high is:

$$
\begin{aligned}
V_{in1} &> V_{in2} \\
V_{ref} / 2 &> V_{DAC} \\
V_{ref} / 2 &> \frac{W}{1024} V_{ref} + V_{ref} / 2 - V_{in} \\
0 &> \frac{W}{1024} V_{ref} - V_{in} \\
\boxed{V_{in} &> \frac{W}{1024} V_{ref}}
\end{aligned}
\tag{8}
$$

Therefore, when Y is high, the trial bit is asserted according to the SAR logic of section 2. X and Y will always be complementary. The comparator outputs X and Y are fed to the R and S inputs respectively of an SR-latch. The Q output of the latch, which tracks Y, is fed into the SAR logic block.

3.5 SAR Logic

3.5.1 Operation

When `start` is asserted, signalling the start of a new conversion, the SAR logic clears its `result` and `cdac_sw` registers, resets the bit index to 9, raises the internal `active` flag, and asserts `cdac_sw[9]`, steering the MSB capacitor to V_{ref}.

On each subsequent rising clock edge, the logic reads `comp_in` — which tracks the comparator output Y — and records the decision in `result[bit_idx]`. The current trial bit in `cdac_sw` is kept if `comp_in` is high, or cleared if low. The next bit is then asserted in `cdac_sw`, updating V_{DAC} for the following comparison as seen in figure 5, and the bit index is decremented. This continues until bit 0 is evaluated, at which point `done` is asserted, `active` is cleared, and the block idles until the next `start` pulse.

3.5.2 Implementation

The SAR logic was written in Verilog RTL and verified behaviourally using Icarus Verilog. It was then synthesised to Sky130 high-density standard cells using Yosys, producing a gate-level SPICE netlist via Yosys's `write_spice` command.

The Yosys-generated netlist required post-processing before it could be used in simulation. Yosys omits power pins (`VDD`/`GND`) from standard cell instantiations, and in some

cases emits zero-volt voltage sources as net aliases rather than explicit connections. A Python script, `fix_spice_ports.py`, was written to correct both issues: it parses the Sky130 PDK SPICE library directly to recover the correct port order for each standard cell, inserts the missing power pins, resolves net aliases, and wraps the result in a `.subckt` block. The output, `sar_logic_fixed.spice`, is the netlist used in all subsequent simulations.

4 Integration & Debugging

With all four blocks verified in isolation (Section 3), the remaining work was to close the loop and find why the full converter did not reproduce the trajectory the behavioural model predicted. Most of the design effort, and the most interesting findings, came from this phase rather than from any single block.

4.1 Full-loop testbench

The integration testbench (`sar_adc_tb_v2.spice`) wires the four blocks into the conversion loop: the CDAC presents `vdac` to the StrongARM comparator, which compares it against `vcm`; the comparator outputs drive a two-NAND SR latch that holds the decision as `comp_in`; and the SAR logic consumes `comp_in` to update the ten switch controls `cdac_sw[9:0]` for the next trial. A single clock edge both captures the previous bit's decision and advances to the next trial, so the loop needs only one clock domain.

Clocking. Each conversion is ten cycles. The testbench runs the clock at 500 kHz (2 µs per cycle, 20 µs per conversion), deliberately relaxed from the 1 MS/s target rate so that `vdac` settles fully on every trial and any error is attributable to the circuit rather than to incomplete settling. Within a cycle, the `cdac_sw` update is followed by ∼990 ns of settling before `clkbar` goes high and the comparator evaluates, leaving ∼1010 ns for the comparator and SR latch to resolve before the next rising clock edge reads `comp_in`.

Cold-start initial conditions. The single most consequential testbench fix was forcing cold-start conditions with `.ic` and the `UIC` flag on `.tran`. Without them, ngspice solves a DC operating point at $t = 0$ with `phi_s` already mid-cycle, which produces a non-physical startup transient on `vdac`: it overshoots to ∼1.32 V (roughly 400 mV above its 0.9 V reset target) and takes hundreds of nanoseconds to settle, corrupting the early trials. The same artifact appeared in two earlier block-pair testbenches before its origin was understood, and it masked the real circuit behaviour across several sessions. The fix initialises `vdac` to its reset value ($V_{\mathrm{cm}} = 0.9\,\mathrm{V}$) and every CDAC bottom plate plus the sampled-input node to V_{in}, matching the state the sample phase will drive them to anyway. The general lesson

— any full-loop transient with a mid-cycle clock state needs explicit initial conditions — is now applied up front rather than rediscovered.

Done-verified readout. Rather than assuming `done` asserts at a fixed time and reading the result there, the testbench measures the actual assertion time with `.meas ... WHEN` and reads `result[9:0]` at 23 µs, with generous margin past the ∼20.5 µs expected assertion. The measured assertion time then serves as a sanity check that the read margin was in fact adequate. Every `cdac_sw` bit and `comp_in` are also probed at full resolution so the bit-by-bit decision sequence can be checked against the precomputed ideal trajectory.

4.2 Isolation-first debugging methodology

The first full-loop run at $V_{\text{in}} = 1.2\,\text{V}$ returned a code roughly 77 LSB below the expected value — a gross error, not a rounding effect. (The reference was later corrected to the true algorithmic target of 682, making the figure −76 LSB; the conclusion that something was seriously wrong is unchanged.)

Rather than debug the closed loop directly, where any of four blocks could be at fault, each block and then each pairwise combination was re-verified in isolation:

- **Switch alone** — sampling accuracy versus input voltage.
- **Comparator alone** — ∼350 µV input kickback, which is expected for a StrongARM topology and benign at this signal level.
- **CDAC alone** — bottom-plate sampling and convert-phase switching.
- **CDAC + comparator chain** — the analog half of the loop with no SAR logic in the path.

This narrowed the error to the sampling switch at high input voltages: the comparator, CDAC, and digital logic were each correct, and the error scaled with V_{in} in a way that pointed at the front-end switch.

4.3 The n-well bootstrap bug

This was the most original finding and the one that took the most work to pin down.

Symptom. The bootstrapped switch undersampled by ∼89 mV at $V_{\text{in}} = 1.5\,\text{V}$, with negligible error at $V_{\text{in}} = 1.2\,\text{V}$ — a strongly input-dependent loss that grew toward the top of the input range.

Wrong hypothesis first. The initial theory was a channel turn-off race on `XQ1`, the PFET that precharges the boost capacitor's top plate. Because `XQ1`'s gate is the very node it is trying to stop charging, an incomplete self-turn-off is a plausible characteristic of this topology. A standalone leakage testbench with a current-sense source in series with `XQ1`'s drain measured a real $\sim$0.2 mA transient at the boost transition, which appeared to confirm the hypothesis.

Falsified by charge balance. The hypothesis did not survive a charge-balance check. Integrating that drain current over its conduction window accounts for only $\sim$1 fC, whereas the charge actually missing from the boost capacitor — measured directly from the precharged-versus- settled voltage across C_{boot} — was $\sim$394 fC. The measured channel current was real but roughly 400$\times$ too small to be the dominant mechanism. Chasing that two-order-of-magnitude gap, rather than accepting a plausible-looking number, is what found the actual bug.

Root cause: forward-biased PMOS body diode. `XQ1` and `XQ2` had their bulk pins tied to fixed V_{DD} — the correct default for ordinary PMOS, but wrong here because these two devices' own source/drain nodes (`net1`, `net3`) are deliberately boosted above V_{DD} during the sample phase. Once a boosted node exceeds V_{DD} by more than a diode drop ($\sim$0.7 V), the source/drain-to-bulk junction stops being reverse biased and conducts as a forward diode — not a small leak but a hard voltage clamp. The signature is unmistakable: `net3` settles at $\sim$2.50 V, almost exactly V_{DD} plus one diode drop. Adding current-sense sources on the bulk pins themselves — a path the drain-side sense source could not see, since bulk connects straight to the V_{DD} rail externally — measured $\sim$382 fC/cycle at $V_{\text{in}} = 1.5$ V, accounting for $\sim$97% of the deficit.

The numbers are self-consistent. The boost node should reach $V_{\text{in}} + V_{\text{DD}} = 3.3$ V at $V_{\text{in}} = 1.5$ V but clamps at 2.5 V, a 0.8 V shortfall. On the 500 fF boost capacitor that corresponds to $0.8\,\text{V} \times 500\,\text{fF} = 400\,\text{fC}$ of charge that never makes it onto the node — within measurement error of the 394 fC found missing.

Fix. A new `nwell_switch` subcircuit generates a floating bias node that always tracks whichever is higher, V_{DD} or `net1` — V_{DD} during precharge, `net1` during boost. The bulk pins of `XQ1` and `XQ2` route through this node instead of fixed V_{DD}, so the junction never forward-biases. The subcircuit's own two internal devices have their bulks tied to its output node, so the fix does not reintroduce the same problem on itself.

Result. In the isolated switch testbench the sampling error dropped from $\sim$89 mV to 0.94 µV, and the bulk-junction current collapsed from -382 fC to $+0.15$ fC per cycle. Re-running the full ADC gave -6, -11, and -54 LSB at $V_{\text{in}} = 0.6$, 1.2, and 1.5 V respectively

— the dominant front-end loss removed, leaving a smaller residual at high input that is the subject of the next subsection.

4.4 Residual error: top-plate reset feedthrough

The -54 LSB at $V_{\text{in}} = 1.5$ V that remains after the bootstrap fix is a separate mechanism. At the sample-to-convert transition, the top-plate reset switch `XTOP_n` couples charge onto `vdac` through its gate-drain overlap capacitance, kicking the node away from its reset value just as the conversion begins.

Isolated testbenches characterised the kick as a function of input voltage:

V_{in} (V)	Kick on `vdac`
0.3	$\sim$0.6 mV
0.6	$\sim$4.8 mV
0.9	$\sim$9 mV
1.2	$\sim$15 mV
1.5	$\sim$122–160 mV (with a $\sim$450–500 ns settling tail)

The kick grows nonlinearly with V_{in} and develops a long settling tail near the top of the range. Because the error is strongly input-dependent rather than a fixed offset, it cannot be removed by a single static digital calibration constant. Known mitigations — a dummy top-plate switch for charge cancellation, or a separate reset phase that lets `vdac` settle before the comparator looks at it — were identified but not implemented, since the converter already meets its functional goals and physical layout is the next priority rather than squeezing the last few LSB of high-input linearity.